

Notre point de départ

Objectif : commencer avec une base commune, très simple, mais déjà fonctionnelle.



Idée clé

On ne commence pas par les fonctionnalités métier. On commence par une baseline technique que toute l'équipe peut lancer, comprendre et modifier.

Mot-clé : BASELINE

Ce que le sujet demande

Le sujet impose certaines briques pour le projet final. Notre méthode de travail vient ensuite.

Frontend + Backend + Database

Obligatoire

Git + commits de tous les membres

Obligatoire

Conteneurisation + lancement en une commande

Obligatoire

.env ignoré + .env.example

Obligatoire

CI GitHub Actions

Pas explicitement obligatoire

/health + DB health check

Notre choix de bootstrap

Conclusion

Notre baseline n'est pas demandée exactement sous cette forme, mais elle respecte et prépare directement les exigences du sujet.

Le repository de départ

ft_transcendence/

├─ frontend/

├─ backend/

├─ database/

├─ docker-compose.yml

├─ .env.example

└─ .github/

└─ workflows/

└─ ci.yml

Au début

Très peu de code.

Seulement ce qui permet de prouver que les composants communiquent.

Déjà opérationnel

Le projet démarre.

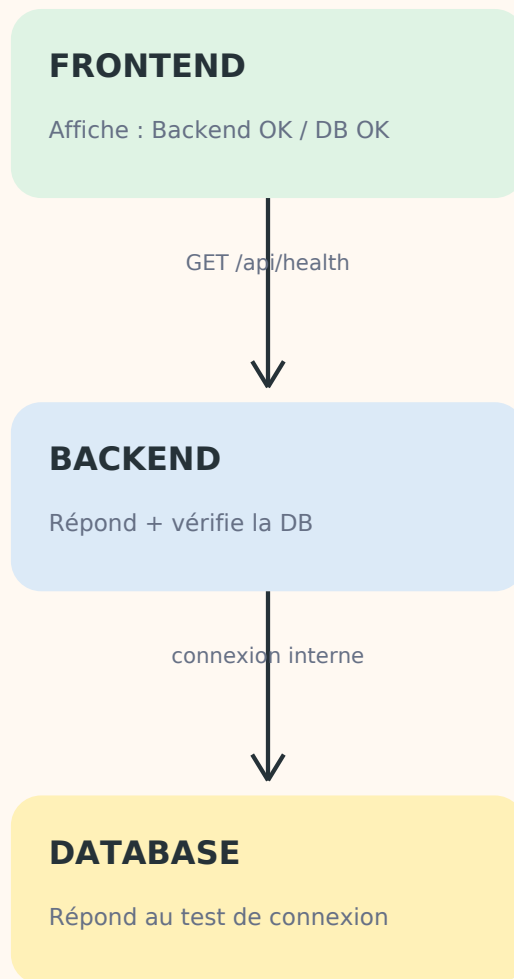
Les conteneurs démarrent.

Les checks répondent.

Pourquoi ?

Parce qu'on veut résoudre les problèmes d'intégration avant que plusieurs features arrivent en parallèle.

Le premier flux à valider

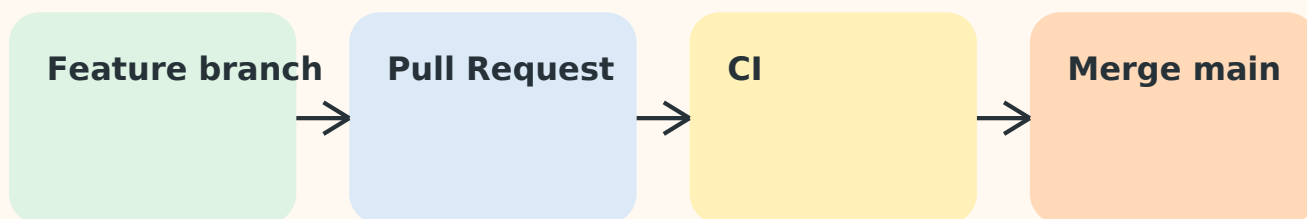


Résultat attendu

Une réponse simple, par exemple : { backend: "ok", database: "ok" }. Ce n'est pas une feature produit : c'est un test de la baseline.

Pourquoi une CI stable ?

Quand plusieurs personnes développent en parallèle, la CI protège la branche principale.



"CI stable" veut dire quoi ?

Pas que le fichier CI ne changera jamais. Cela veut dire : avant les vraies features, le pipeline de base est fiable et donne un signal clair.

✓ **Build front**

✓ **Build back**

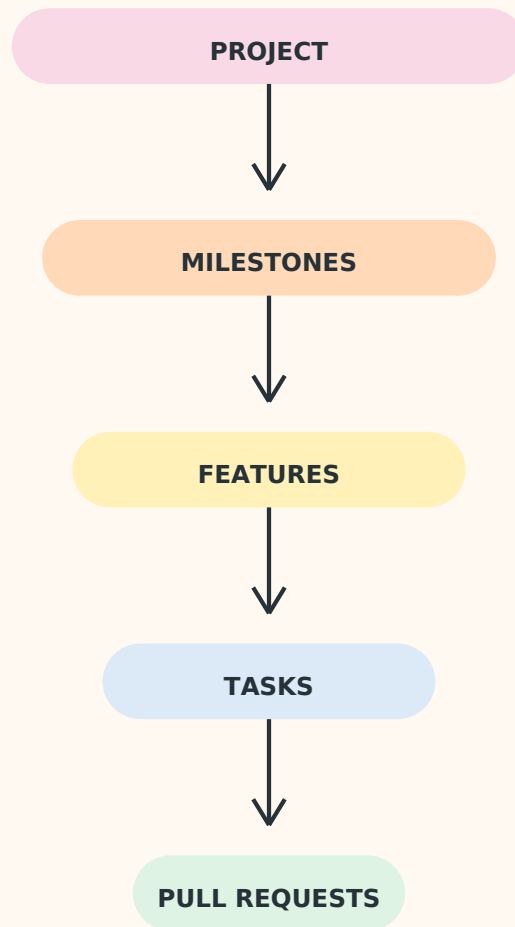
✓ **Build images**

✓ **Start stack**

✓ **Health check**

✓ **Tests**

Comment on travaille ensuite ?



Vocabulaire simple

Feature = une fonctionnalité visible du produit.

Task = un morceau de travail concret nécessaire pour réaliser la feature.

On ne divise pas l'équipe par technologie

À éviter

Personne A = Front
Personne B = Back
Personne C = Database
Personne D = DevOps

Notre approche

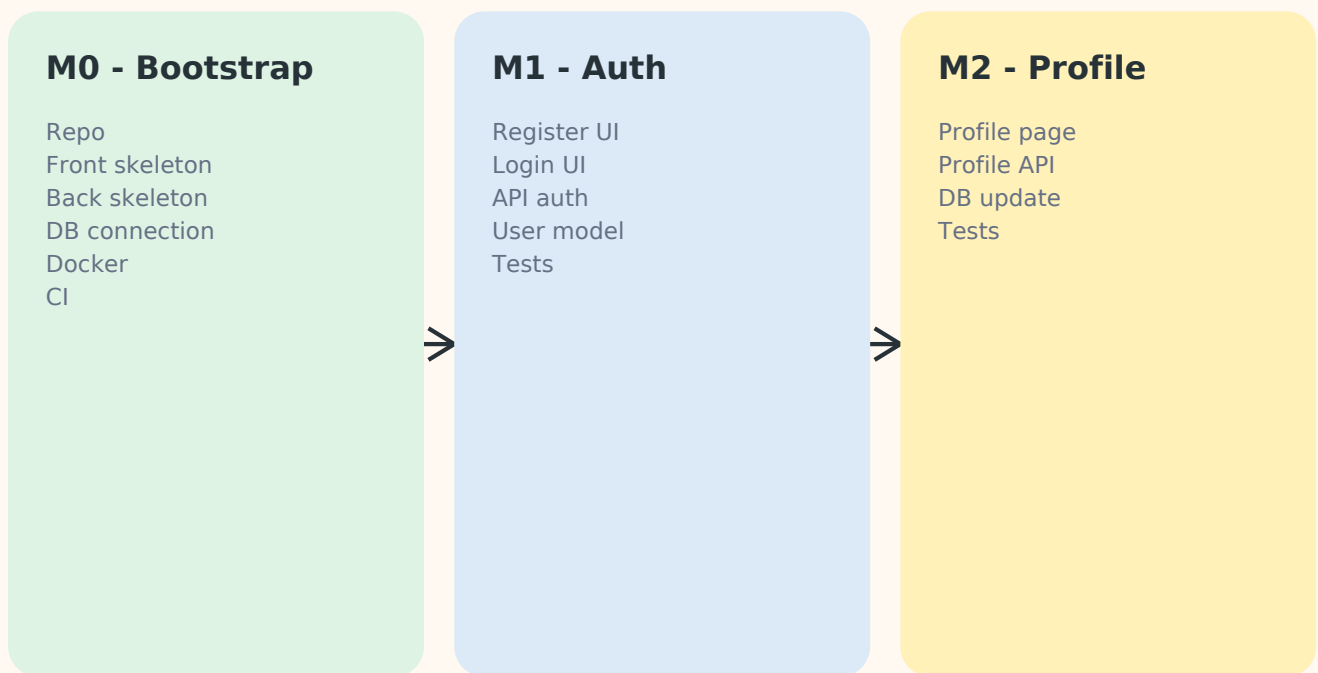
On distribue des features / tasks.
Une personne peut toucher le front, le back
et la DB.

Exemple : Feature Authentication

Task 1 : formulaire frontend
Task 2 : API login/register
Task 3 : modèle User en DB
Task 4 : tests
Task 5 : adaptation CI si nécessaire

Mot-clé : VERTICAL SLICE

Exemple de roadmap



Règle de progression

On ajoute une feature, la CI la valide, puis on merge. main doit rester notre référence stable.

Le message à présenter à l'équipe

Présentation en 45 secondes

On commence par une baseline commune.

Le repository contient le frontend, le backend et la database, déjà conteneurisés. Au début, il n'y a presque aucune logique métier : le frontend vérifie seulement que le backend répond, et le backend vérifie que la database est disponible.

On met ensuite en place une CI fiable pour valider automatiquement cette baseline.

Une fois la base stable, on définit les features, on les découpe en tasks, puis chacun prend des tasks. Une personne peut donc travailler sur le frontend, le backend et la database pour une même fonctionnalité.

Chaque changement passe par une Pull Request et doit garder la CI verte avant d'être intégré dans main.

Baseline → Features → Tasks → PR → CI → Main

Les mots-clés à retenir

BASELINE

Base technique commune et fonctionnelle

END-TO-END

Front → Back → Database

CI STABLE

Signal fiable avant chaque merge

FEATURE

Fonctionnalité produit

TASK

Travail concret à réaliser

VERTICAL SLICE

Une feature peut traverser front/back/DB

PULL REQUEST

Point de review et de validation

MAIN STABLE

Référence commune de l'équipe